

Rygar Sentinel: A Unified Edge Appliance for Secure AIoT Infrastructure Management

Official Build & Engineering Project Report

AIoT Design Challenge 2026

**Submitted by
Developer: SIVASINGAM K**

Developer Name

Sivasingam K
EMBEDDED SYSTEM DEVELOPER & ENGINEER

Team Members & Roles

S. No.	Name	Role & Primary Technical Contribution	Contact / Identifier
1	Sivasingam K	System Architecture, Hardware Engineering, Embedded C/C++ Firmware, Python & Flask Backend, AIoT Anomaly Pipeline, Testing & Documentation	sivasingam403@gmail.com

Project Submission Checklist

Deliverable Item	Verification Detail
Project Title	Rygar Sentinel: A Unified Edge Appliance for Secure AIoT Infrastructure Management
DIGIKEY Complete Bill of Materials (BOM)	Categorized list covering all 25+ hardware & software modules
Detailed Project Description	Step-by-step structural, logical, and application breakdown
Complete Schematic Diagrams	Pin-level interconnect topologies for all sub-nodes
Complete Prototype Firmware & Source Code	Arduino sketches, ESP32/Heltec wireless communication firmware, Python/Flask services, MQTT bridge, dashboard, CCTV and AI inference code.
CAD Enclosure / Physical Layout	Model-office multi-room architectural physical layout
Project Photographs	High-resolution physical prototype snapshots

Deliverable Item	Verification Detail
System Operation Demonstration Video	Live functional test video recorded and linked
Technical Explanation Video	Full architectural walk-through



Video Demonstrations

[VIDEO DEMONSTRATION AND EXPLANATION LINK](#)

Component List (Bill of Materials)

[LINK FOR DIGIKEY BOM](#)

S.No	Component Name	Manufacturer	Part	DigiKey Part Number	Qty
.		Number			

1	Arduino UNO Q, 2GB RAM, 16GB eMMC	ABX00162	1050-ABX00162-ND	1
2	Raspberry Pi 5, 8GB RAM	SC1432	2648-SC1432-ND	1
3	Pironman 5 Mini PC Case	CN0446D	4411-CN0446D-ND	1
4	Raspberry Pi 27W USB-C Power Supply	SC1158	2648-SC1158-ND	1
5	32GB MicroSD Card, Class 10	COM-19041	1568-COM-19041-ND	1
6	128GB M.2 NVMe SSD	IM2P41E8-128GCTB5	4670-IM2P41E8-128GCTB5-ND	1
7	DHT11 Temperature & Humidity Sensor	SNS-DH11	1188-SNS-DHT11-ND	2
8	RC522 RFID Module	CN0090	4411-CN0090-ND	1
9	HC-SR501 PIR Motion Sensor	ST0012	4411-ST0012-ND	2
10	12-Key Numeric Keypad	419	1528-1136-ND	1
11	ACS712 20A Current Sensor	ACS712ELCTR-20A-T	620-1190-1-ND	1
12	Arduino UNO R3	A000066	1050-1024-ND	1
13	ESP32-WROOM-32E Development Board	ESP32-DEVKITC-32E	1965-ESP32-DEVKITC-32E-ND	2
14	0.96" OLED Graphic Display	326	1528-1220-ND	1
15	16x2 Character LCD with I ² C	AMC1602AR-B-Y6WFDY-I2C	5491-AMC1602AR-B-Y6WFDY-I2C-ND	1
16	MQ135 Air Quality Sensor	SNS-MQ135	1188-SNS-MQ135-ND	1

17	MQ-2 Gas Sensor Module	ST0252	4411-ST0252-ND	1
18	Analog Flame Sensor	DFR0076	1738-1091-ND	1
19	IR Obstacle Avoidance Sensor	ST0050	4411-ST0050-ND	2
20	Reed Switch	MK04-1A66B-500W	374-1115-ND	1
21	5V Magnetic Buzzer	WT-1205	433-1028-ND	1
22	4-Channel Relay Module	TS0011	4411-TS0011-ND	1
23	Grove Water Sensor	101020018	1597-1127-ND	2
24	40×10mm 12V DC Cooling Fan	HA40101V4-1000U-A99	259-1790-ND	1
25	Male-to-Male Jumper Wires, 6"	1957	1528-1967-ND	1
26	Male-to-Female Jumper Wires, 6"	1954	1528-1964-ND	1
27	General-Purpose Breadboard	1609	1528-1195-ND	1
28	RC Servo Motor, 4.8V	SER0006	1738-1385-ND	1

Software Layer:

Software Layer	Component Name	Technical Purpose & Functionality
Firmware	Arduino IDE / C++	Embedded control logic, sensor/actuator interfacing, hardware control, and serial data transmission.
Communication	Eclipse Mosquitto MQTT	Lightweight publish/subscribe messaging broker used for transferring unified system state between the Arduino UNO Q and Raspberry Pi 5.
Edge Compute	Python 3.x Runtime	Execution environment for Flask services, MQTT bridging, data

Software Layer	Component Name	Technical Purpose & Functionality
		processing, dashboard services, and AI inference on the edge systems.
Web Service	Flask Web Framework	Lightweight HTTP server providing REST-style endpoints for node communication, system state, and the web dashboard.
Computer Vision	OpenCV	Webcam capture and video-frame processing for the prototype CCTV stream.
Analytics & AI	NumPy, Pandas, Scikit-Learn	Sensor-data processing, feature preparation, normalization/scaling, and Isolation Forest-based anomaly detection.
Model Persistence	Joblib	Serialization and loading of the trained machine-learning preprocessing/model components for local inference.
Private Storage	Nextcloud / SQLite	Nextcloud provides private local file/data storage, while SQLite can be used for local event/telemetry persistence.
Remote Access	Tailscale	Private encrypted overlay networking for remote access to the locally hosted dashboard and services without exposing them directly to the public Internet.

Introduction & Problem Statement

The rapid adoption of connected devices, sensors, surveillance systems, access-control systems, and automated infrastructure has transformed modern offices, schools, homes, and small facilities into increasingly complex cyber-physical environments. However, these

technologies are often deployed as separate systems, with individual devices collecting and processing information independently. This creates fragmented infrastructure in which security, environmental monitoring, occupancy, safety, automation, data storage, and surveillance are not always able to operate as a unified system.

Rygar Sentinel is proposed as a **Unified Edge Appliance for AIoT Infrastructure Management** that brings these distributed functions together at the local edge. Instead of treating each sensor or subsystem as an independent application, the platform provides a common architecture through which different devices can collect information, communicate system states, identify unusual conditions, and support appropriate responses.

The prototype combines **embedded controllers, wireless sensor nodes, edge computing, MQTT communication, local data processing, machine-learning-based anomaly detection, physical actuators, CCTV streaming, private storage, and secure remote access**. An **Arduino UNO Q** acts as the central edge coordinator for the distributed embedded nodes, while a **Raspberry Pi 5** serves as the Sentinel Core responsible for higher-level processing, MQTT messaging, dashboard services, AI inference, and integration of the different subsystems.

The system monitors multiple aspects of a physical environment, including **access events, door status, temperature, humidity, gas and air-quality conditions, water presence, flame conditions, occupancy, parking status, electrical load activity, and controllable devices such as lighting and a DC fan**. These inputs are brought together into a unified system state that can be viewed through a centralized dashboard.

A key aspect of Rygar Sentinel is its **local-first architecture**. Core monitoring and safety functions are designed to operate within the local infrastructure rather than depending entirely on cloud services. Critical hardware-level responses, such as activating a buzzer when a flame condition is detected, remain local to the sensor node. At the same time, the Raspberry Pi provides machine-learning-based anomaly detection that examines combinations of system-state features to identify conditions that deviate from the learned normal operating baseline.

The platform also integrates a prototype **CCTV stream** and provides **secure remote access through Tailscale**, allowing authorized users to access the locally hosted monitoring interface without directly exposing the system to the public Internet. Private local storage and supporting services can further extend the appliance into a broader edge infrastructure platform.

Therefore, Rygar Sentinel is not designed as a single-purpose automation device. It demonstrates a **modular AIoT edge architecture** in which sensing, communication, computation, monitoring, physical control, anomaly detection, surveillance, storage, and remote access can operate as parts of a common system. The prototype provides a foundation that can be adapted for environments such as **small offices, educational**

institutions, clinics, homes, small businesses, and remote facilities, where locally managed and integrated infrastructure can be valuable.

Absolutely bro. For **Detailed Project Description**, we should go deeper than the introduction and describe the **entire working system**, not just the entrance subsystem.

2. Detailed Project Description

Rygar Sentinel is a **Unified Edge Appliance for Secure AIoT Infrastructure Management** designed to integrate heterogeneous sensors, embedded controllers, physical actuators, surveillance, local computing, artificial intelligence, data management, and secure remote access into a single edge-oriented platform.

The system is designed around the principle of:

Connect → Observe → Collect → Correlate → Detect → Explain → Act → Store

Rather than operating individual IoT devices as isolated systems, Rygar Sentinel creates a common infrastructure in which distributed nodes collect information from the physical environment and forward their system states to an edge coordination layer. The collected information is subsequently processed by the Raspberry Pi 5 Sentinel Core, where monitoring, communication, AI-based anomaly detection, and visualization are performed.

2.1 Distributed Sensing and Control

The physical environment is divided into multiple functional nodes. The prototype contains an **Entrance Access-Control Node** and an **Edge Sensor Node**, allowing different categories of infrastructure information to be collected simultaneously.

The Entrance Node combines an Arduino UNO, keypad, reed switch, servo motor, buzzer, and ESP32 communication module. It provides PIN-based access control while also monitoring the physical state of the door. Access events such as successful authentication, failed authentication, door opening, and door closing are transmitted from the embedded controller to the wireless communication layer.

The Edge Sensor Node provides broader environmental and infrastructure monitoring. It combines an Arduino UNO with multiple sensors and actuators, including:

- MQ-2 gas/smoke sensor
- MQ-135 air-quality sensor
- DHT11 temperature and humidity sensor
- Flame sensor
- Water sensor
- PIR occupancy sensors

- IR-based parking detection
- ACS712 current sensor
- Buzzer
- Relay-controlled lighting
- 12V DC fan

This allows the system to observe different dimensions of the physical environment rather than relying on a single sensor category.

2.2 Wireless Edge Communication

The distributed embedded nodes use ESP32-based wireless communication to transfer their collected information to the **Arduino UNO Q**.

The architecture separates physical sensing from higher-level coordination. The Arduino UNO controllers handle direct sensor and actuator interaction, while the ESP32/Heltec communication layer provides wireless connectivity.

The UNO Q therefore acts as the central coordinator for the distributed embedded nodes. It receives information from the entrance and edge sensor subsystems and provides a unified system state to the Raspberry Pi 5.

2.3 Arduino UNO Q Edge Coordinator

The **Arduino UNO Q** forms the intermediate edge-coordination layer of Rygar Sentinal.

Its role is to bridge the distributed embedded nodes and the Raspberry Pi 5 Sentinel Core. The UNO Q receives the system information generated by the connected nodes, maintains the current system state, and provides a central interface for the connected infrastructure.

A 16×2 I²C LCD connected to the UNO Q provides a local display for system information. This allows important system states to be presented locally without requiring the user to access the web dashboard.

The UNO Q also exposes HTTP endpoints through its local Flask-based service for receiving information from the wireless nodes. The resulting state is subsequently bridged to the Raspberry Pi using MQTT.

2.4 Raspberry Pi 5 Sentinel Core

The **Raspberry Pi 5** functions as the main computing and integration platform of the system.

It provides the higher-level services required by Rygar Sentinal, including:

- MQTT messaging
- System-state processing
- Web dashboard
- AI anomaly detection
- CCTV integration
- Local data services
- Remote-access integration

The Raspberry Pi therefore acts as the **Sentinel Core**, while the Arduino UNO Q remains responsible for coordinating the distributed embedded nodes.

This separation allows resource-intensive software functions to run on the Raspberry Pi without placing unnecessary computational requirements on the microcontroller nodes.

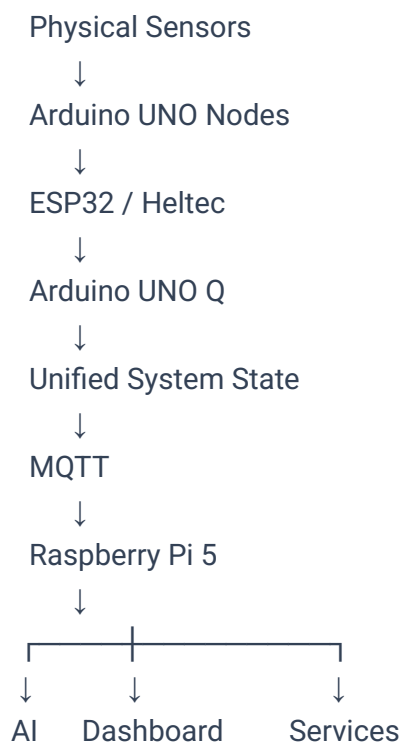
2.5 MQTT-Based System Communication

Eclipse Mosquitto is used as the MQTT broker on the Raspberry Pi.

The unified state generated through the UNO Q is published through MQTT using the Rygar Sentinal system topic. The Raspberry Pi dashboard and AI engine can subscribe to this common state stream.

This provides a lightweight publish/subscribe communication mechanism between the different software components.

The resulting logical data path is:



2.6 Centralized Monitoring Dashboard

Rygar Sentinal provides a web-based monitoring dashboard hosted on the Raspberry Pi 5.

The dashboard provides a centralized view of the distributed infrastructure, allowing the operator to observe information such as:

- Temperature
- Humidity
- Gas sensor readings
- Air-quality sensor readings
- Water sensor status
- Flame/fire status
- Door status
- Access events
- Occupancy status
- Parking sensor status
- Lighting status
- Fan status
- Current-sensor readings
- CCTV stream

This transforms individual sensor readings into a unified operational view of the monitored environment.

2.7 Local AI-Based Anomaly Detection

An important component of Rygar Sentinal is the integration of **machine-learning-based anomaly detection**.

Instead of using AI to replace deterministic safety controls, the system uses AI to analyze combinations of system-state variables and identify conditions that differ from the learned normal operating pattern.

The current implementation uses:

- NumPy
- Pandas
- Scikit-learn
- StandardScaler
- Isolation Forest
- Joblib

A baseline dataset is collected from normal system operation. The collected sensor-state features are then processed and used to train an Isolation Forest model.

During operation, incoming system states are passed through the same preprocessing pipeline and evaluated by the trained model.

The output provides an indication such as:

NORMAL

or

ANOMALY DETECTED

The AI therefore provides an additional layer of situational awareness over the distributed infrastructure.

2.8 Local Safety and Physical Response

Rygar Sentinel maintains a distinction between **AI-based anomaly detection** and **deterministic safety responses**.

Critical safety responses are implemented directly at the embedded node. For example, when the flame sensor detects a fire condition, the Arduino UNO can activate the local buzzer without waiting for the Raspberry Pi, cloud services, or AI inference.

Similarly, sensor conditions can be used to control physical devices through the relay module. The prototype demonstrates relay-based control of room lighting and a low-voltage 12V DC fan.

This architecture reduces dependence on network availability for basic local safety functions.

2.9 CCTV Integration

A laptop webcam is used as the prototype CCTV source.

The camera is accessed through **OpenCV**, while a lightweight Flask service provides the video stream to the Rygar Sentinel dashboard.

The CCTV component provides visual monitoring alongside the sensor-based infrastructure data.

In the current prototype, CCTV is primarily a **visual monitoring subsystem**. The machine-learning anomaly detector operates on the structured sensor/system-state data rather than claiming computer-vision-based AI detection.

2.10 Private Data and Remote Access

The platform can be extended with local/private data services such as **Nextcloud** for locally managed storage.

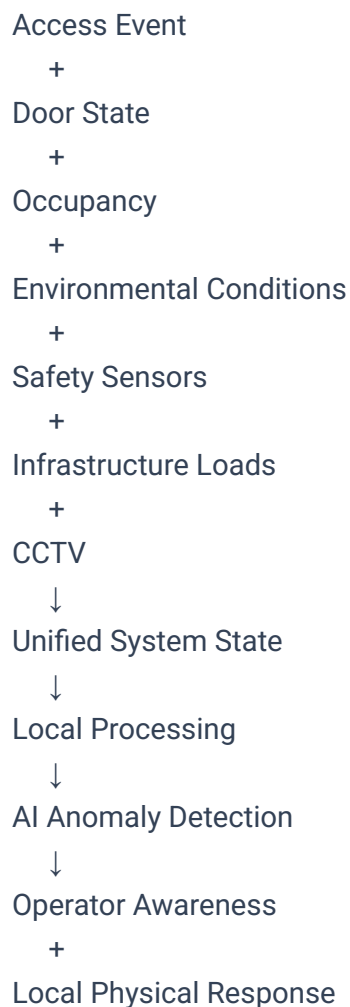
Remote access is implemented through **Tailscale**, allowing authorized devices to access the locally hosted dashboard through a private encrypted network rather than exposing the dashboard directly to the public Internet.

This enables the operator to access the system from a mobile device while maintaining the local-first architecture.

2.11 Unified Operating Concept

The major objective of Rygar Sentinal is not simply to connect more sensors. The platform demonstrates how different infrastructure functions can participate in a common edge-computing architecture.

For example, the system can simultaneously observe:



This architecture allows the prototype to demonstrate a pathway from **individual sensors and embedded controllers to an integrated AIoT edge appliance**.

2.12 Prototype Scope



The current implementation is a functional prototype intended to demonstrate the architecture, integration, and feasibility of the concept. It is not presented as a production-certified building-management or safety system.

The prototype demonstrates the integration of:

Embedded sensing + wireless communication + edge coordination + MQTT + Raspberry Pi computing + centralized monitoring + local AI anomaly detection + physical actuation + CCTV + secure remote access.

This modular architecture allows additional sensors, nodes, applications, and AI models to be incorporated in future versions without fundamentally changing the core Sentinel architecture.

Yes bro — for the **Schematics** section, don't just write “see attached schematic.” The report should explain what each schematic represents and how the circuits relate to the overall architecture.

Since the actual two schematic PDFs are the source, we should preserve what they show rather than inventing additional circuitry. SENTINAL - ENTRANCE node 2

3. Schematics

The hardware design of Rygar Sentinal is divided into two principal embedded subsystems: the **Entrance Access-Control Node** and the **Edge Sensor Node**. The corresponding electrical schematics define the controller interfaces, sensor connections, actuator connections, communication interfaces, and power connections used in the prototype.

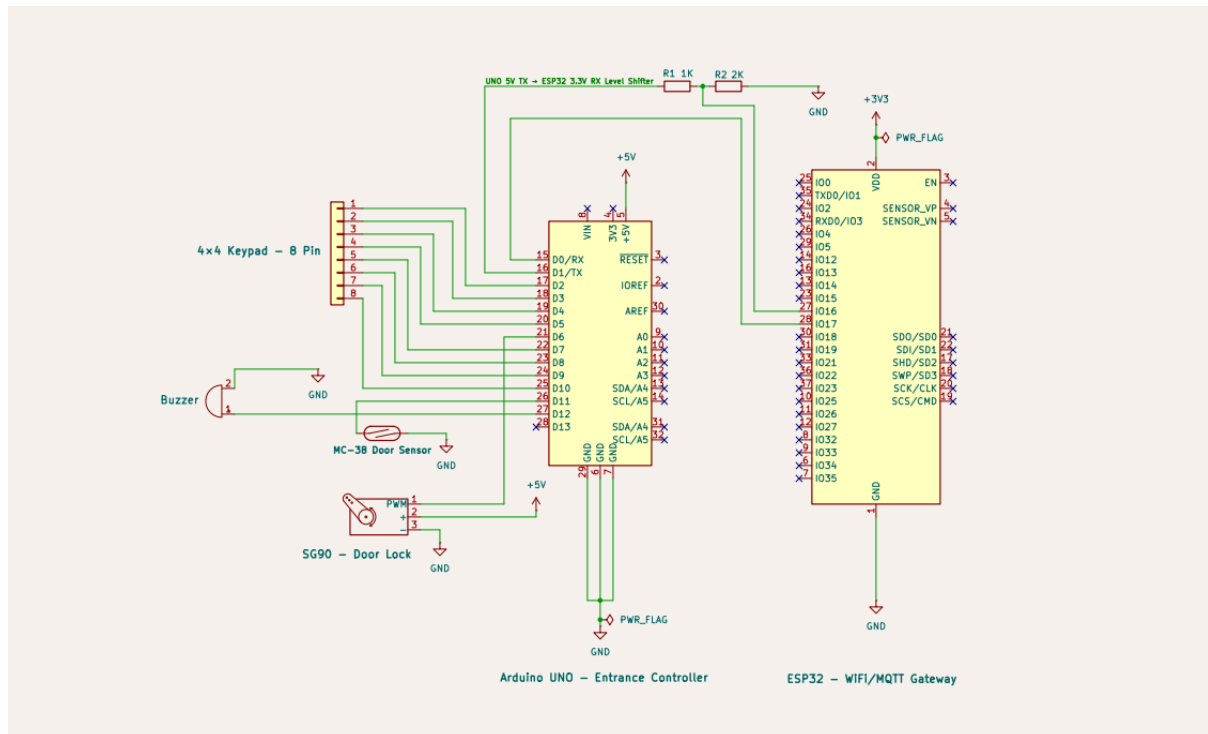
3.1 Entrance Access-Control Schematic

The Entrance Access-Control schematic contains an **Arduino UNO**, **4×4 keypad**, **MC-38 reed switch**, **SG90 servo motor**, **buzzer**, and an **ESP32 wireless communication module**. The circuit is designed to provide both access authentication and physical door-state monitoring. SENTINAL - ENTRANCE

The Arduino UNO performs the local access-control operations. The keypad provides the user input for PIN authentication, while the SG90 servo is used as the physical locking mechanism. The MC-38 reed switch provides feedback regarding the physical door state, allowing the controller to distinguish between door-open and door-closed conditions.

The buzzer provides local audible feedback for access and alarm-related events.

The Arduino UNO communicates with the ESP32 through a UART interface. Because the Arduino UNO operates at a 5 V logic level while the ESP32 uses 3.3 V logic, the schematic includes a **1 kΩ / 2 kΩ resistor voltage-divider interface** on the UNO transmit line to provide a suitable logic level for the ESP32 receiver.



The schematic therefore represents the **physical access-control and communication boundary** of the Entrance Node.

3.2 Edge Sensor Node Schematic

The Edge Sensor Node schematic contains an **Arduino UNO R3** connected to multiple environmental, occupancy, safety, infrastructure-monitoring, and actuator components. The schematic also includes an **ESP32-WROOM-32 communication interface** and a **DHT11 temperature/humidity sensor**. node 2

The analog sensing section includes:

- MQ-2 gas/smoke sensor
- MQ-135 air-quality sensor
- ACS712 current sensor
- Water sensor

Digital sensing includes:

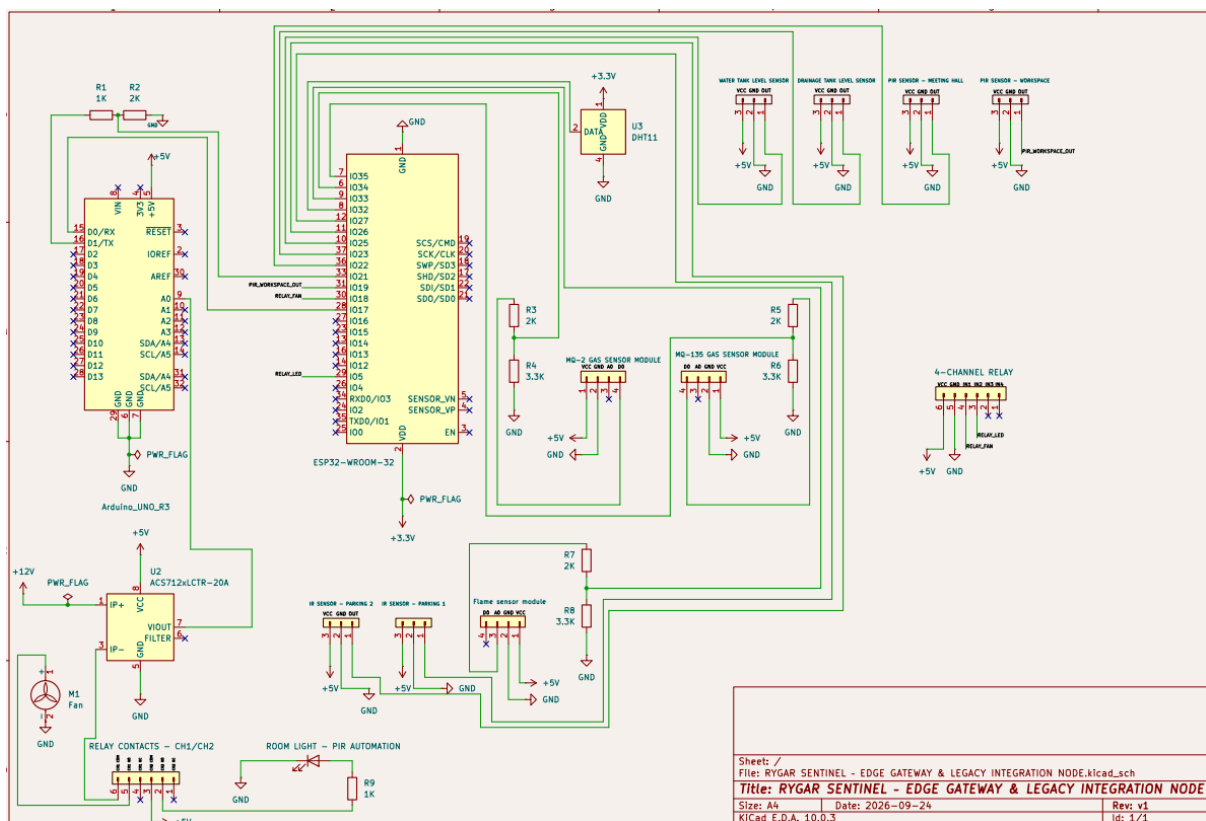
- PIR motion sensors
- IR sensors for parking detection
- Flame sensor

The actuator section contains:

- Buzzer
- 4-channel relay module
- 12 V DC fan
- Lighting load

The Arduino UNO reads the sensor inputs and controls the connected actuators according to the programmed operating logic. The ACS712 is positioned in the low-voltage DC load path to provide an electrical-load/current observation for the prototype. The 12 V DC fan is used as the demonstration load rather than connecting the system directly to mains electricity.

The logical structure of the Edge Sensor Node is:



3.3 Controller-to-Communication Interface

Both embedded nodes use a separation between **local hardware control** and **wireless communication**.

The Arduino UNO is responsible for direct interaction with sensors and actuators, while the ESP32-based communication layer transfers the resulting system information wirelessly to the Arduino UNO Q.

This separation allows the sensor/actuator circuitry to remain relatively independent from the higher-level network architecture.

3.4 Electrical Design Considerations

The schematics also establish several important electrical design considerations:

1. **Logic-level compatibility**

The UART connection between the 5 V Arduino UNO and 3.3 V ESP32 uses a resistor-divider interface on the UNO transmit path.

2. **Common ground reference**

The connected communication and peripheral circuits require an appropriate common ground reference.

3. **Low-voltage actuator operation**

The prototype uses a 12 V DC fan as the demonstrated load and does not directly switch 230 V mains equipment.

4. **Relay isolation**

The relay module provides the switching interface between the controller logic and the low-voltage demonstration loads.

5. **Local safety response**

The flame-detection path is connected to the Arduino control logic so that the buzzer can be activated locally.

3.5 Schematic-to-System Relationship

The two schematics represent the **physical hardware layer** of Rygar Sentinal. They are connected to the higher-level software architecture as follows:

CONNECT: Arduino UNO controllers process the connected sensors and actuators. The ESP32/Heltec communication nodes receive structured data from the UNO controllers and transmit the information wirelessly to the Arduino UNO Q.

COORDINATE: The Arduino UNO Q receives information from the distributed nodes, maintains the current unified system state, and provides the state to the Raspberry Pi 5 through the MQTT bridge.

ANALYZE: The Raspberry Pi 5 receives the unified system state through MQTT. The AI engine processes selected system-state features using a trained Isolation Forest anomaly-detection model.

DECIDE: Deterministic safety logic remains at the embedded node. For example, flame detection activates the local buzzer. The AI layer separately identifies unusual combinations of system-state values.

ACT: Local embedded controllers operate the buzzer, servo lock, lighting relay, and low-voltage fan relay according to the implemented control logic.

MONITOR: The unified system state is presented through the Raspberry Pi dashboard, while the AI training dataset and trained model are maintained locally during prototype development.

Practical Implementation Scenario: The Intelligent Office

- **Zone A – Entrance**

A user enters a PIN through the keypad. A valid PIN causes the SG90 servo to unlock the door temporarily, while an invalid PIN generates an access-denied event and buzzer feedback. The magnetic reed switch independently monitors the physical door state.

- **Zone B – Occupancy and Lighting**

PIR sensors monitor activity in the monitored rooms. When motion is detected, the Arduino controller activates the lighting relay. The current prototype uses this mechanism to demonstrate occupancy-based lighting control.

- **Zone C – Environmental and Safety Monitoring**

The Edge Sensor Node continuously monitors gas, air-quality, temperature, humidity, water, and flame-related sensor states. Flame detection can directly activate the local buzzer without requiring the Raspberry Pi or cloud connectivity.

- **Infrastructure Load Monitoring**

The ACS712 is used to observe the electrical load associated with the low-voltage demonstration fan. The prototype treats the sensor output as a current/load observation rather than a calibrated energy-metering system.

- **Core Compute**

The Arduino UNO Q coordinates the distributed embedded nodes, while the Raspberry Pi 5 provides MQTT messaging, centralized monitoring, AI inference, CCTV integration, and remote dashboard access.

Integrated Automation Decision Trees

[Occupancy Automation]

PIR Motion Triggered → Room State = OCCUPIED → Relay 1 ON (Lights) → Start 300s Timeout Timer

No Motion Detected → Timer Expired → Relay 1 OFF (Energy Saving Mode)

[Safety Intervention]

Gas / Flame Sensor HIGH → Trip Isolation Relay OFF → Sound Siren BZ2 → Push Urgent Dashboard Alert

[Entrance Security Pipeline]

PIN Input Matrix → Valid Code? —YES— → Actuate Servo Gate OPEN → Wait 5s → Servo CLOSE

|

NO → Increment Failed Counter → Counter >= 3? → Sound BZ1 → Log Security Event

Why This Represents Genuine Edge AIoT

Standard commercial IoT devices simply forward sensor data to a third-party cloud server where processing occurs, introducing network latency, security vulnerabilities, and reliance on active internet connectivity.

Rygar Sentinel executes **true local AIoT architecture**:

- **On-Device Feature Extraction:** Multi-sensor feature scaling happens locally at the edge.
- Rygar Sentinel demonstrates an edge-oriented AIoT architecture by combining distributed embedded sensing, local communication, edge coordination, centralized local computing, and machine-learning-based anomaly detection without requiring cloud-based processing for the core prototype workflow.

- **Local Data Processing:** Sensor information is collected by distributed embedded nodes and transferred through the Arduino UNO Q to the Raspberry Pi 5.
- **Local AI Inference:** The Raspberry Pi 5 runs the trained Isolation Forest anomaly-detection model locally using incoming system-state data.
- **Deterministic Local Safety:** Critical hardware-level responses, such as flame detection activating the local buzzer, remain within the embedded node and do not depend on AI inference or Internet connectivity.

Code

The Rygar Sentinel software is implemented as a distributed set of embedded firmware and Python-based edge applications. The code is divided according to the functional responsibilities of each hardware and software layer.

The implementation follows the system architecture in which the Arduino-based nodes handle direct sensor and actuator operations, ESP32-based controllers provide wireless communication, the Arduino UNO Q coordinates the distributed nodes, and the Raspberry Pi 5 provides higher-level services such as MQTT communication, dashboard visualization, CCTV integration, and AI-based anomaly detection.

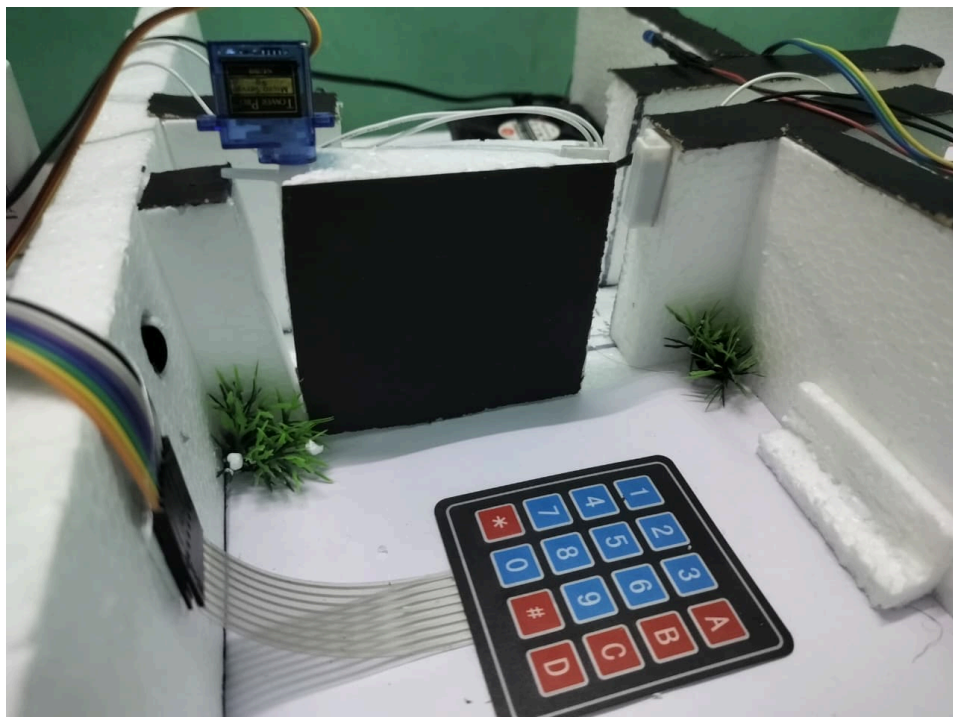
Firmware Architecture

The embedded firmware is developed using **Arduino C/C++**. Each controller has a dedicated program according to its role in the system.

Source File	Platform	Purpose
<code>entrance_uno.ino</code>	Arduino UNO	Keypad-based access control, servo door lock, reed-switch monitoring, and buzzer control

<code>entrance_esp32.ino</code>	ESP32	Receives entrance events from the Arduino UNO and forwards them wirelessly to the UNO Q
<code>edge_node_uno.ino</code>	Arduino UNO	Reads environmental, occupancy, safety, parking, and current-sensing inputs and controls local actuators
<code>heltec_v1.ino</code>	Heltec WiFi LoRa 32 V1	Receives edge-node data, reads DHT11 data, and transfers the combined information to the UNO Q
<code>node2_server.py</code>	Arduino UNO Q	Provides the central HTTP service for receiving node data and maintaining the unified system state

Entrance Controller



The Entrance Arduino firmware implements:

- 4x4 keypad scanning
- PIN authentication
- Servo-based door locking
- Reed-switch door-state monitoring
- Buzzer feedback
- Serial event transmission

The firmware generates events such as:

ENTRANCE_ONLINE
ACCESS_GRANTED
ACCESS_DENIED
DOOR_OPEN
DOOR_CLOSED

These events are transferred to the ESP32 communication layer.

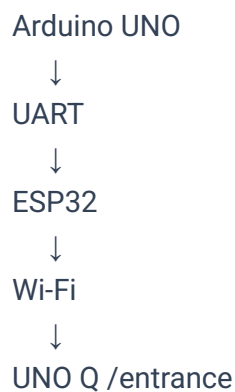
Entrance ESP32

The Entrance ESP32 firmware provides the wireless communication layer between the Entrance Arduino UNO and the Arduino UNO Q.

The program:

1. Initializes the UART interface.
2. Connects to the local Wi-Fi network.
3. Reads events transmitted by the Arduino UNO.
4. Converts the event into a JSON message.
5. Sends the message to the UNO Q HTTP endpoint.

The logical flow is:



Edge Sensor Node Firmware

The Edge Sensor Node Arduino firmware is responsible for acquiring multiple sensor readings and controlling local outputs.

The firmware reads:

- MQ-2

- MQ-135
- ACS712
- Water sensor
- PIR sensors
- Parking IR sensors
- Flame sensor

It also controls:

- Buzzer
- Room lighting relay
- Fan relay

The node periodically generates a structured data record containing the sensor and actuator states.

Example:

```
NODE2,MQ2=66,ACS=517,WATER=90,MQ135=73,
PIR1=0,PIR2=0,PARK1=1,PARK2=1,
FLAME=1,FIRE=0,BUZZER=0,FAN=0,LIGHT=0
```

This information is transmitted through the UART interface to the Heltec/ESP32 communication layer.

Heltec V1 Communication Firmware

The Heltec WiFi LoRa 32 V1 acts as the wireless communication controller for the Edge Sensor Node.

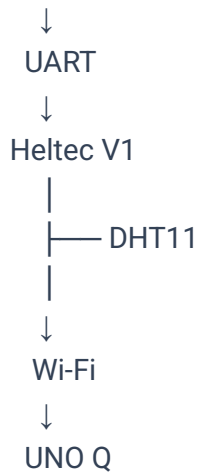
In addition to forwarding the Arduino UNO sensor data, it reads the DHT11 temperature and humidity sensor.

The resulting data is packaged into a JSON structure containing:

- Node identification
- Temperature
- Humidity
- Arduino sensor-state data

The data is then transmitted over Wi-Fi to the UNO Q.

Edge Arduino UNO



Arduino UNO Q Software

The Arduino UNO Q provides the central coordination layer between the distributed wireless nodes and the Raspberry Pi 5.

The Python-based Flask service provides endpoints including:

/node2
/entrance
/state
/

The `/entrance` endpoint receives entrance events, while `/node2` receives environmental and infrastructure information.

The `/state` endpoint provides the current unified system state to downstream services.

The UNO Q also interfaces with the centralized **16×2 I²C LCD**, allowing system information to be displayed locally.

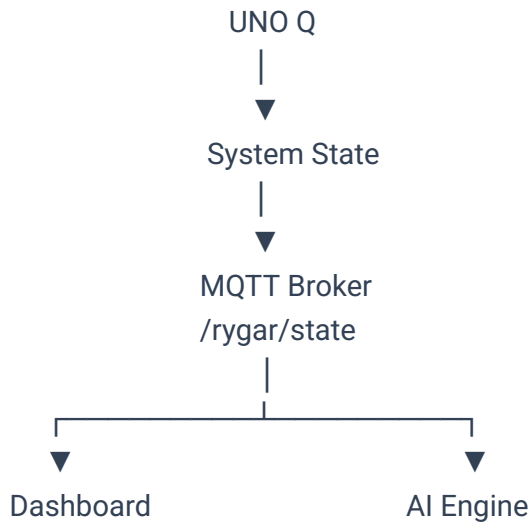
MQTT Communication

The Raspberry Pi 5 runs the **Mosquitto MQTT broker**.

The unified system state from the UNO Q is published to:

rygar/state

The MQTT architecture allows multiple services to consume the same system-state stream without requiring each service to communicate directly with the embedded nodes.

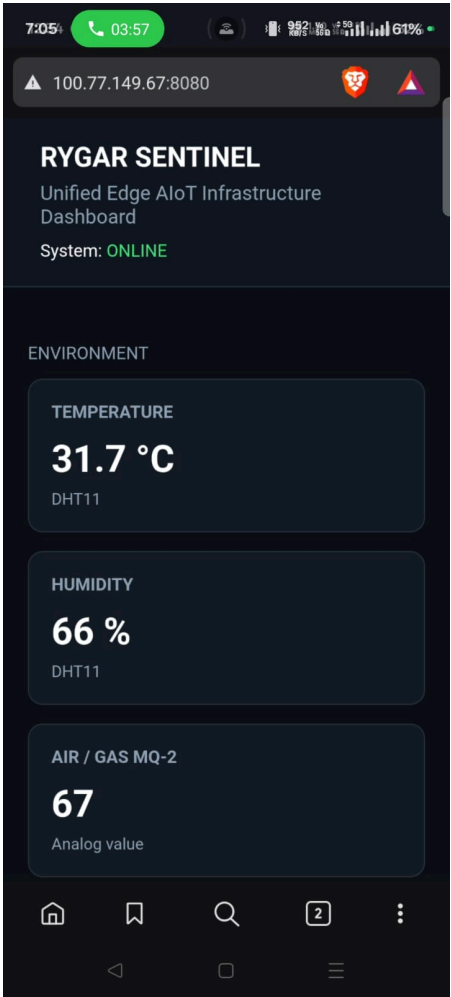
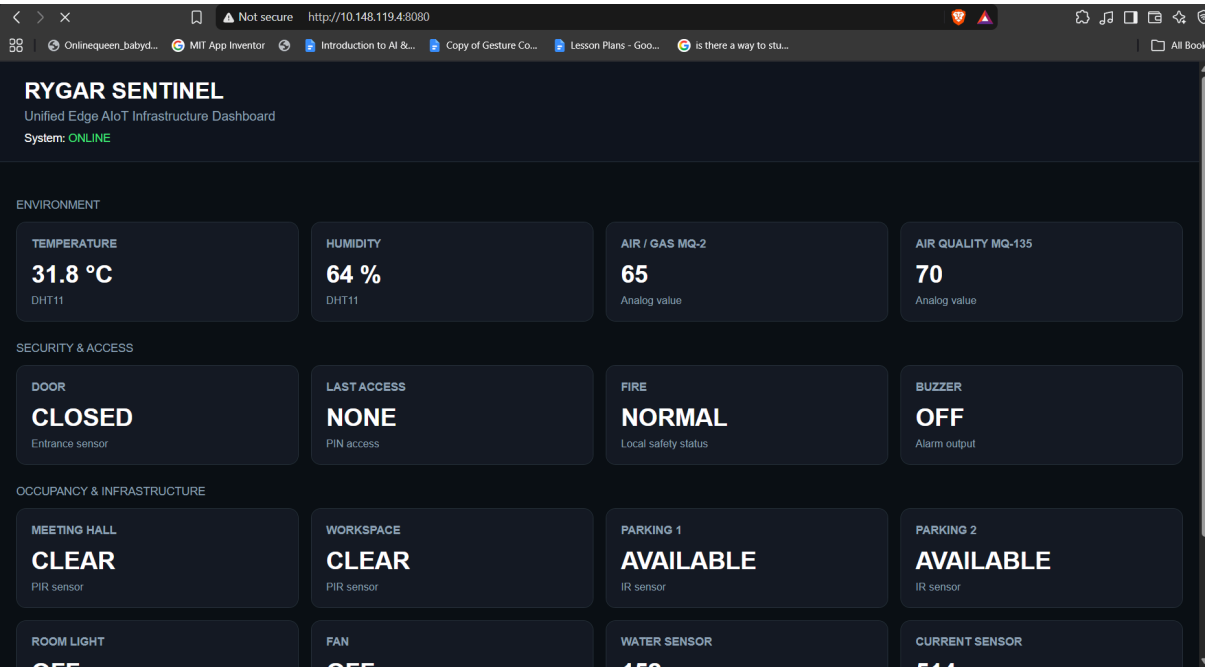


The MQTT bridge periodically retrieves the UNO Q state and publishes it to the Raspberry Pi MQTT broker.

Web Dashboard

The dashboard is implemented using **Python Flask** and provides a centralized interface for monitoring the Rygar Sentinel system.

The dashboard receives system-state information through MQTT and presents the information in a browser-based interface.



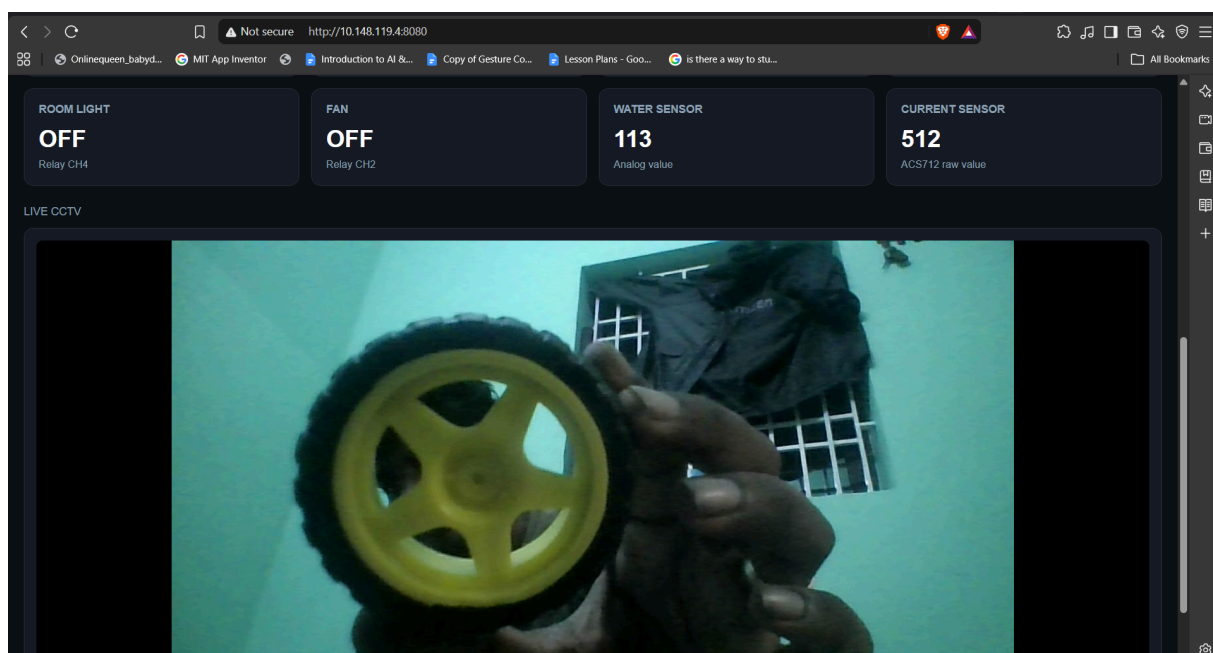
The interface includes monitoring information such as:

- Temperature
- Humidity
- MQ-2
- MQ-135
- Water level
- Door state
- Access events
- Flame/fire state
- Occupancy
- Parking
- Lighting
- Fan
- Current-sensor readings
- CCTV stream

The dashboard therefore provides a single operational view of the distributed system.

CCTV Software

The prototype CCTV service is implemented using **Python and OpenCV**.



The program:

1. Opens the laptop webcam.
2. Continuously captures frames.
3. Encodes frames as JPEG images.
4. Provides an MJPEG stream through Flask.
5. Embeds the stream into the Rygar Sentinel dashboard.

The CCTV implementation is used for **visual monitoring**. It is not represented as the source of the project's machine-learning anomaly detection.

AI and Machine Learning Code

The AI component is implemented on the Raspberry Pi 5 using:

- NumPy
- Pandas
- Scikit-learn
- Joblib

The AI implementation is divided into three stages.

```
=====
RYGAR SENTINEL - LIVE AI ENGINE
=====
AI model loaded
AI scaler loaded
Listening to: rygar/state

/home/sivasingam/rygar-ai/ai_engine.py:176: DeprecationWarning: Callback API version 1 is deprecated, update to latest
ersion
  client = mqtt.Client(client_id="rygar_ai_engine")
AI ENGINE: MQTT CONNECTED
AI ENGINE: SUBSCRIBED
[18:42:52] ● SENTINEL AI: NORMAL (score=0.060)
[18:42:52] ● SENTINEL AI: ANOMALY DETECTED
          Situation: unusual combination of sensor readings
          AI score: -0.053
[18:42:54] ● SENTINEL AI: NORMAL (score=0.030)
[18:42:56] ● SENTINEL AI: NORMAL (score=0.019)
[18:42:58] ● SENTINEL AI: NORMAL (score=0.065)
[18:43:00] ● SENTINEL AI: NORMAL (score=0.019)
```

Data Collection

`ai_collector.py` subscribes to the MQTT system-state stream and records normal operating conditions.

MQTT State



Feature Extraction



Normal Dataset

Model Training

`train_model.py` prepares the collected data and trains an **Isolation Forest** anomaly-detection model.

The preprocessing pipeline includes feature scaling before model training.

Normal Dataset



Feature Preparation



StandardScaler



Isolation Forest



Trained Model

Live Inference

`ai_engine.py` subscribes to the live MQTT state and applies the trained model to incoming system states.

The inference result is reported as either:

NORMAL

or:

ANOMALY DETECTED

The AI therefore evaluates the **combined system state**, rather than treating one sensor reading as an independent AI decision.

Software Repository Organization

All source code is maintained in the project GitHub repository and organized according to the hardware and software architecture.

firmware/

├── entrance_uno/

├── entrance_esp32/

- └─ edge_node_uno/
- └─ heltec_v1/
- └─ uno_q/

- └─ raspberry_pi/
 - └─ mqtt/
 - └─ dashboard/
 - └─ cctv/

- └─ ai/
 - └─ collector/
 - └─ training/
 - └─ inference/

This organization separates embedded firmware from edge-computing services and AI software, making the system easier to maintain and extend.

Code Availability

The complete source code used for the prototype is maintained in the project's GitHub repository:

RYGAR SENTINAL – AIoT Design Challenge 2026

The repository contains the embedded firmware, Raspberry Pi services, AI implementation, hardware documentation, schematics, and project documentation.

Code Implementation and Configuration

The complete prototype source code is maintained in the project GitHub repository and is organized according to the system architecture.

The embedded firmware consists of separate programs for the Entrance Arduino UNO, Entrance ESP32, Edge Sensor Arduino UNO, and Heltec V1 communication node. These programs handle sensor acquisition, actuator control, local safety logic, serial communication, and wireless communication.

The Arduino UNO entrance controller uses the keypad, servo, reed switch, and buzzer to implement local access-control functionality. The Edge Sensor Arduino reads the environmental, safety, occupancy, parking, water, and current-sensing inputs and controls the connected relay and buzzer outputs.

The ESP32 and Heltec V1 programs provide wireless communication between the embedded nodes and the Arduino UNO Q.

The UNO Q runs the central Flask-based state service and exposes endpoints for receiving entrance and sensor-node data. A separate MQTT bridge publishes the unified state to the Mosquitto broker running on the Raspberry Pi 5.

The Raspberry Pi software contains separate applications for MQTT communication, dashboard visualization, CCTV streaming, AI data collection, model training, and live anomaly inference.

For security, Wi-Fi credentials, passwords, and other private configuration values are not included in the public source repository.

Connection Explanation & Circuit Interfaces

Node A: Entrance Access Control Sub–System (Arduino Uno R3)

Component	Arduino UNO Pin	Function
Keypad Row 1	D2	Keypad input
Keypad Row 2	D3	Keypad input
Keypad Row 3	D4	Keypad input
Keypad Row 4	D5	Keypad input

SG90 Servo	D6	Door-lock control
Keypad Column 1	D7	Keypad interface
Keypad Column 2	D8	Keypad interface
Keypad Column 3	D9	Keypad interface
Keypad Column 4	D10	Keypad interface
Reed Switch	D11	Door-state monitoring
Buzzer	D12	Audible indication
UART TX	D1	Communication to ESP32 through level shifting
UART RX	D0	Optional / not relied upon in current communication flow

Node B: Environmental, Safety & Power Automation (Arduino Uno R3)

Component	Arduino UNO Pin	Function
MQ-2 Gas Sensor	A0	Gas/smoke sensing
ACS712	A1	Current/load observation
Water Sensor	A2	Water detection
MQ-135	A3	Air-quality sensing
PIR — Meeting Hall	D2	Occupancy detection
PIR — Workspace	D3	Occupancy detection
Parking IR 1	D4	Parking detection
Parking IR 2	D5	Parking detection
Flame Sensor	D6	Flame detection

Buzzer	D7	Local safety alarm
Relay CH4	D12	Lighting control
Relay CH2	D13	12 V fan control
UART TX	D1	Communication to Heltec V1

Circuit Schematics & Electrical Layout

The electrical design is documented through separate schematics for the Entrance Access-Control Node and Edge Sensor Node.

The prototype uses low-voltage electrical interfaces for the demonstrated loads. The 12 V DC fan is used as the demonstration load through the relay module, and the system does not directly switch 230 V mains equipment.

The UART interfaces between the 5 V Arduino UNO and 3.3 V ESP32/Heltec devices use the resistor-divider level-shifting arrangement documented in the schematics.

The complete schematics and pin mappings are included in the project repository under:

```
hardware/  
├── bom/  
├── pin_maps/  
└── schematics/
```

Project Video Demonstration Structure

[VIDEO DEMONSTRATION AND EXPLANATION LINK](#)

Project Video Demonstration Structure

The submitted demonstration video follows the major functional layers of the prototype:

1. Architectural Overview

Display of the physical model-office structure showing the Entrance Node, Edge Sensor Node, Arduino UNO Q, Raspberry Pi 5, and camera setup.

2. Access Control Walkthrough

Live demonstration of valid PIN entry triggering the servo lock, invalid PIN feedback, and magnetic door-state monitoring.

3. Environmental and Safety Monitoring

Demonstration of the Edge Sensor Node, including gas, air-quality, temperature/humidity, water, occupancy, parking, flame, and current/load sensing.

4. Centralized Monitoring

Demonstration of the UNO Q coordinating node information and the Raspberry Pi dashboard displaying the unified system state.

5. Edge AI Anomaly Detection

Demonstration of the Raspberry Pi AI engine processing combined system-state data and identifying anomalous operating patterns.

6. CCTV and Remote Access

Demonstration of the webcam stream in the dashboard and remote dashboard access through the private Tailscale network.

Full Source Code Repository & Firmware Structure

RYGAR-SENTINAL-FOR-AIOT-DESIGN-CHALLENGE-2026/

```
|— firmware/  
|— raspberry_pi/  
|— ai/  
|— hardware/  
|— docs/  
|— images/
```

GitHub Source Code Repository

RYGAR SENTINAL – AIoT Design Challenge 2026

Repository:

<https://github.com/Sivasingam2004/RYGAR-SENTINAL-FOR-AIOT-DESIGN-CHALLENGE-202>

6

GitHub Source Code Repository Link

- Public GitHub Repository: [GITHUB LINK](#)

Summary & Conclusion

Summary

Rygar Sentinel is a prototype **Unified Edge Appliance for Secure AIoT Infrastructure Management** that integrates distributed sensing, access control, physical actuation, edge computing, centralized monitoring, CCTV, local AI-based anomaly detection, private storage, and secure remote access within a common architecture.

The prototype combines **Arduino UNO-based embedded nodes, ESP32 wireless communication, an Arduino UNO Q edge coordinator, and a Raspberry Pi 5 Sentinel Core**. The distributed nodes collect information from the physical environment and transfer their states through the UNO Q to the Raspberry Pi, where the information can be monitored and analyzed centrally.

The implemented prototype demonstrates multiple infrastructure-monitoring functions, including:

- PIN-based entrance access control
- Door-state monitoring
- Temperature and humidity monitoring
- Gas and air-quality sensing
- Water sensing
- Flame detection
- Occupancy monitoring
- Parking detection
- Current/load observation
- Relay-based lighting and fan control
- Local buzzer-based safety response
- Centralized web dashboard
- CCTV streaming
- MQTT-based system-state communication

- Machine-learning-based anomaly detection
- Secure remote dashboard access through Tailscale

The system follows a **local-first edge architecture**, where core sensing, communication, monitoring, and selected safety functions can operate within the local infrastructure without making cloud connectivity a fundamental requirement.

The AI component provides an additional layer of system intelligence by learning a baseline of normal operating conditions and using an **Isolation Forest** model to identify unusual combinations of system-state parameters. This allows the platform to move beyond simple individual sensor monitoring toward broader infrastructure-state analysis.

Conclusion

The development of Rygar Sentinal demonstrates the feasibility of integrating heterogeneous embedded systems and infrastructure-monitoring functions into a unified AIoT edge platform.

The project establishes a complete flow from **physical sensing and control to wireless communication, edge coordination, centralized processing, visualization, and local AI-based anomaly detection**. The use of the Arduino UNO Q as an intermediate edge coordinator and the Raspberry Pi 5 as the Sentinel Core provides a modular separation between embedded hardware control and higher-level computing services.

A significant design principle of the prototype is the separation between **deterministic local safety actions and AI-based anomaly detection**. Safety-critical responses such as flame-triggered buzzer activation remain within the local embedded system, while the AI layer provides additional situational awareness by identifying unusual system-state patterns.

The prototype also demonstrates that an edge appliance can combine several traditionally separate functions—such as access control, environmental sensing, occupancy monitoring, surveillance, automation, and remote monitoring—within a common infrastructure.

Although the current implementation is a prototype and requires further development for production deployment, it provides a foundation for future expansion into environments such as **offices, schools, clinics, homes, small businesses, and remote facilities**. Future versions could incorporate additional sensor nodes, stronger authentication, improved data persistence, more advanced AI models, industrial-grade hardware, improved fault tolerance, and professionally engineered enclosures and power systems.

Overall, **Rygar Sentinal demonstrates a practical approach to building a modular, local-first AIoT infrastructure platform in which connected devices are not treated as isolated components, but as coordinated elements of a unified edge system.**